

Тема урока: модификаторы доступа и аксессоры

1. Модификаторы доступа
2. Методы аксессоры (геттеры и сеттеры)

Аннотация. Урок посвящен модификаторам доступа и аксессорам.

Модификаторы доступа

Ранее мы упоминали, что доступ к атрибутам объекта должны иметь только методы этого объекта. Такой подход защищает атрибуты от случайного повреждения. Однако во всех рассмотренных ранее классах атрибуты не являлись защищенными, ведь внешний код с легкостью мог получить к ним доступ .

В классических языках программирования (C++, Java, C#) доступ к атрибутам реализуется с помощью ключевых слов `public` (публичный), `protected` (защищенный) и `private` (приватный):

- публичные атрибуты доступны для работы вне класса
- доступ к защищенным атрибутам возможен только внутри текущего класса, а также внутри унаследованных от него классов
- приватные атрибуты недоступны извне — с ними можно работать только внутри текущего класса

С точки зрения разграничения доступа к атрибутам Python является особым языком — в нем отсутствует механизм, который мог бы запретить внешнему коду взаимодействовать с атрибутами объекта или класса. Вместо этого создатели Python предложили следующий ряд соглашений:

- если имя атрибута начинается с одного нижнего подчеркивания (`_name`), то он считается **защищенным**
- если имя атрибута начинается с двух нижних подчеркиваний (`__name`), то он считается **приватным**

В Python все атрибуты являются публичными по умолчанию. Любой атрибут объекта может быть доступен за пределами класса, независимо от того, является он публичным, защищенным или приватным.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.name = name

cat = Cat('Кемаль')
print(cat.name)

cat.name = 'Роджер'
print(cat.name)
```

ВЫВОДИТ:

```
Кемаль
Роджер
```

Как мы видим, никаких проблем не возникает ни при обращении к атрибутам, ни при их изменении. Аналогичное поведение наблюдается и с защищенными атрибутами, то есть атрибутами, имя которых начинается с одного символа нижнего подчеркивания.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self._name = name

cat = Cat('Кемаль')
print(cat._name)

cat._name = 'Роджер'
print(cat._name)
```

ВЫВОДИТ:

```
Кемаль
Роджер
```

Несколько иначе себя ведут приватные атрибуты. Если мы предварим имя атрибута двумя нижними подчеркиваниями, то есть сделаем его приватным, то код за пределами класса не сможет получить к нему доступ напрямую.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.__name = name

cat = Cat('Кемаль')

print(cat.__name)
```

приводит к возбуждению исключения:

```
AttributeError: 'Cat' object has no attribute '__name'
```

На первый взгляд может показаться, что для внешнего кода приватного атрибута действительно не существует, однако если мы посмотрим на содержимое словаря атрибутов объекта, то увидим что на самом деле приватный атрибут лишь получил другое имя.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.__name = name

cat = Cat('Кемаль')

print(cat.__dict__)
```

выводит:

```
{'__Cat__name': 'Кемаль'}
```

Дело в том, что, делая атрибут приватным, на самом деле мы лишь неявно изменяем его имя. Такое поведение называется **искажением имени**.



Любой атрибут вида `__name` текстуально заменяется на `__class__name`, где `class` — это имя текущего класса.

Таким образом, возможность обратиться к приватному атрибуту, а также изменить его значение все же остается.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.__name = name

cat = Cat('Кемаль')

cat._Cat__name = 'Роджер'

print(cat.__dict__)
```

ВЫВОДИТ:

```
{'_Cat__name': 'Роджер'}
```

Примечания

Примечание 1. Хорошая статья про именование с подчеркиванием в Python доступна по [ссылке](https://django.fun/ru/articles/python/naming-underscores-python/) (https://django.fun/ru/articles/python/naming-underscores-python/).

Примечание 2. Физически механизм ограничения доступа к атрибутам в Python реализован слабо, лишь на уровне соглашения, поэтому ответственность за соблюдение данного соглашения ложится на плечи программистов. Защищенные атрибуты помогают указать, что их изменение может нарушить логику работы класса, но не накладывают строгих ограничений.

Приватные атрибуты используются для более жесткой инкапсуляции. Программист как бы предупреждает, что не следует использовать эти атрибуты, так как они являются служебными и могут быть изменены без предварительного предупреждения.

Примечание 3. Искажение имени приватного атрибута происходит лишь при его установке внутри класса.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.__name = name

cat = Cat('Кемаль')

cat.__age = 1

print(cat.__dict__)
```



